



Escuela de Ingeniería en Computación
Ingeniería en Computación
IC6821 - Diseño de Software

Proyecto: Diseño de Arquitectura

Plataforma para la Enseñanza de Programación

Mishelle Orellana Jiménez
m.orellana.1@estudiantec.cr
2024099469

Amanda Montero Zúñiga
amontzun@estudiantec.cr
2021579522

San José, Costa Rica
Abril 2026

Índice general

1. Introducción	2
1.1. Explicación del problema	2
1.2. Explicación de la solución	2
1.3. Justificación de la solución	2
2. Desarrollo	4
2.1. Diseño de Persistencia	4
2.1.1. 2.1 Modelo Conceptual	4
2.1.2. 2.2 Modelo Lógico en 4FN	7
2.2. Diseño de Arquitectura	17
2.2.1. 2.1 Diagrama de Componentes	17
2.2.2. 2.2 Diagrama de Clases	19
3. Conclusiones	20
Referencias	23

Capítulo 1

Introducción

1.1. Explicación del problema

En cursos introductorios de programación se presentan dos retos principales: la verificación de la autoría real del código y la trazabilidad de los cambios realizados por los estudiantes. Con el aumento de herramientas externas de generación de código, se vuelve más difícil asegurar que las entregas reflejen el aprendizaje individual. Además, la revisión manual consume tiempo y no siempre detecta alteraciones o reutilización indebida de soluciones.

El diseño de una plataforma académica para la enseñanza de programación requiere una arquitectura que permita separar responsabilidades, facilitar el mantenimiento y permitir la evolución del sistema. En este sentido, el uso de principios de diseño de software y patrones orientados a objetos permite construir soluciones más reutilizables, extensibles y comprensibles para futuros desarrolladores (Gamma, Helm, Johnson, y Vlissides, 1994).

1.2. Explicación de la solución

Se propone una plataforma académica con una IDE especializada que integra validación criptográfica de archivos `.py`, control de versiones con Git y ejecución de pruebas automáticas. La arquitectura separa responsabilidades por capas, lo que permite mantener componentes independientes para autenticación, gestión de asignaciones, verificación de integridad y persistencia. Esta separación facilita el mantenimiento y mejora la claridad del flujo de información.

1.3. Justificación de la solución

La solución fortalece la integridad académica al validar la autenticidad de los archivos y registrar cambios de manera confiable. También reduce el tiempo de evaluación al automatizar pruebas y centralizar la gestión de entregas. El uso de

patrones de diseño y un modelo de datos normalizado promueve escalabilidad, reutilización y adaptación a futuras necesidades del curso.

Capítulo 2

Desarrollo

2.1. Diseño de Persistencia

El modelo de persistencia se diseñó siguiendo principios del modelo relacional, con el objetivo de representar las entidades principales del sistema y sus relaciones de forma clara y consistente. La normalización de las tablas permite reducir redundancias, evitar anomalías de actualización y mantener la integridad de los datos almacenados (Silberschatz, Korth, y Sudarshan, 2019).

2.1.1. 2.1 Modelo Conceptual

El modelo conceptual representa las entidades principales del sistema y sus relaciones. La plataforma está orientada a cursos introductorios de programación, donde los docentes publican asignaciones, los estudiantes realizan entregas y el sistema registra evidencia de integridad, trazabilidad y resultados de evaluación automática.

El diagrama Entidad-Relación permite representar de manera visual las entidades del dominio, sus atributos principales y las relaciones existentes entre ellas. Este tipo de diagrama facilita la comprensión inicial del sistema antes de transformar el diseño conceptual en un modelo lógico relacional (Mermaid, 2026).

Las entidades identificadas para el sistema son las siguientes:

Cuadro 2.1: Entidades principales del modelo conceptual

Entidad	Descripción
Rol	Define el tipo de permisos o función que puede tener un usuario dentro del sistema, como estudiante, docente o administrador.
Usuario	Representa a cualquier persona que utiliza la plataforma. Cada usuario posee un rol asociado.
Curso	Representa un curso académico donde se publican asignaciones y se matriculan usuarios.
Matrícula	Relaciona usuarios con cursos, indicando la participación de cada usuario dentro de un curso.
Asignación	Representa una tarea, práctica o proyecto creado por un docente dentro de un curso.
Entrega	Representa la entrega realizada por un estudiante para una asignación específica.
Archivo	Representa los archivos de código fuente enviados como parte de una entrega.
Validación Hash	Almacena el resultado de la validación criptográfica aplicada a un archivo.
Commit Git	Registra la información de los commits asociados a una entrega.
Caso de Prueba	Representa los casos de prueba definidos para evaluar una asignación.
Resultado de Prueba	Almacena el resultado obtenido al ejecutar un caso de prueba sobre una entrega.

Cardinalidad y participación

La cardinalidad permite identificar cuántas instancias de una entidad pueden relacionarse con otra. La participación indica si la existencia de una entidad depende obligatoriamente de otra dentro de la relación.

Cuadro 2.2: Cardinalidad y participación del modelo conceptual

Relación	Cardinalidad	Participación
Rol – Usuario	Un rol puede estar asociado a muchos usuarios. Cada usuario debe tener un rol.	Rol parcial, Usuario total.
Usuario – Matrícula	Un usuario puede estar matriculado en muchos cursos. Cada matrícula pertenece a un usuario.	Usuario parcial, Matrícula total.
Curso – Matrícula	Un curso puede tener muchas matrículas. Cada matrícula pertenece a un curso.	Curso parcial, Matrícula total.
Curso – Asignación	Un curso puede tener muchas asignaciones. Cada asignación pertenece a un curso.	Curso parcial, Asignación total.
Usuario – Asignación	Un docente puede crear muchas asignaciones. Cada asignación es creada por un docente.	Usuario parcial, Asignación total.
Usuario – Entrega	Un estudiante puede realizar muchas entregas. Cada entrega pertenece a un estudiante.	Usuario parcial, Entrega total.
Asignación – Entrega	Una asignación puede tener muchas entregas. Cada entrega pertenece a una asignación.	Asignación parcial, Entrega total.
Entrega – Archivo	Una entrega puede contener uno o más archivos. Cada archivo pertenece a una entrega.	Entrega total, Archivo total.
Archivo – Validación Hash	Un archivo puede tener una validación hash. Cada validación pertenece a un archivo.	Archivo parcial, Validación Hash total.
Entrega – Commit Git	Una entrega puede tener uno o varios commits asociados. Cada commit pertenece a una entrega.	Entrega parcial, Commit Git total.
Asignación – Caso de Prueba	Una asignación puede tener varios casos de prueba. Cada caso de prueba pertenece a una asignación.	Asignación parcial, Caso de Prueba total.
Entrega – Resultado de Prueba	Una entrega puede tener muchos resultados de prueba. Cada resultado pertenece a una entrega.	Entrega parcial, Resultado de Prueba total.
Caso de Prueba – Resultado de Prueba	Un caso de prueba puede ejecutarse muchas veces. Cada resultado corresponde a un caso de prueba.	Caso de Prueba parcial, Resultado de Prueba total.

Diagrama Entidad-Relación

El diagrama Entidad-Relación del sistema se muestra en la Figura 2.1. Este diagrama representa las entidades principales del dominio, sus atributos relevantes, las relaciones entre ellas y la cardinalidad correspondiente.

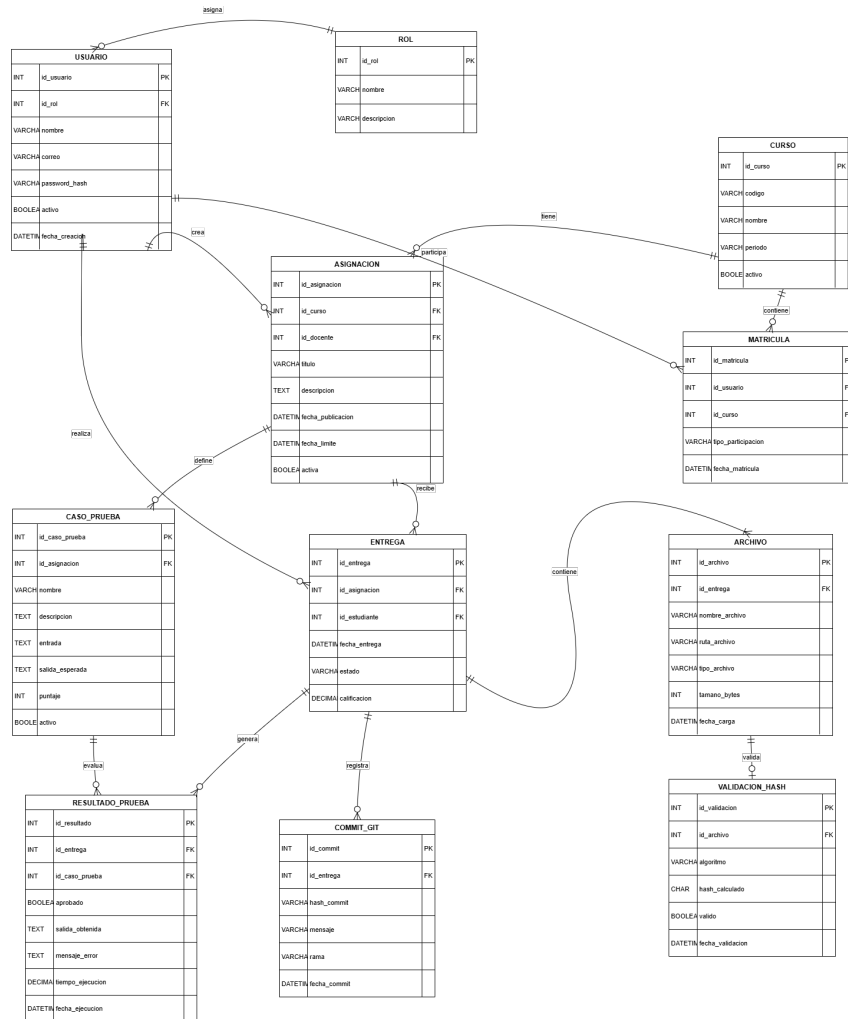


Figura 2.1: Diagrama Entidad-Relación del sistema

2.1.2. 2.2 Modelo Lógico en 4FN

El modelo lógico transforma el modelo conceptual en un conjunto de tablas relacionales normalizadas. Para este diseño se aplica normalización hasta la Cuarta Forma Normal, con el objetivo de eliminar redundancias, evitar dependencias multivaluadas y asegurar que cada tabla represente una única entidad o relación del sistema.

En este modelo, los archivos, validaciones criptográficas, commits, casos de prueba y resultados de prueba se separan en tablas independientes. Esto permite registrar múltiples archivos por entrega, múltiples commits asociados, varios casos de prueba por asignación y diferentes resultados de ejecución sin duplicar

información dentro de las tablas principales.

Diagrama Relacional normalizado

El modelo relacional normalizado se muestra en la Figura 2.2. En este diagrama se representan las tablas, sus claves primarias, claves foráneas y las relaciones normalizadas entre ellas.

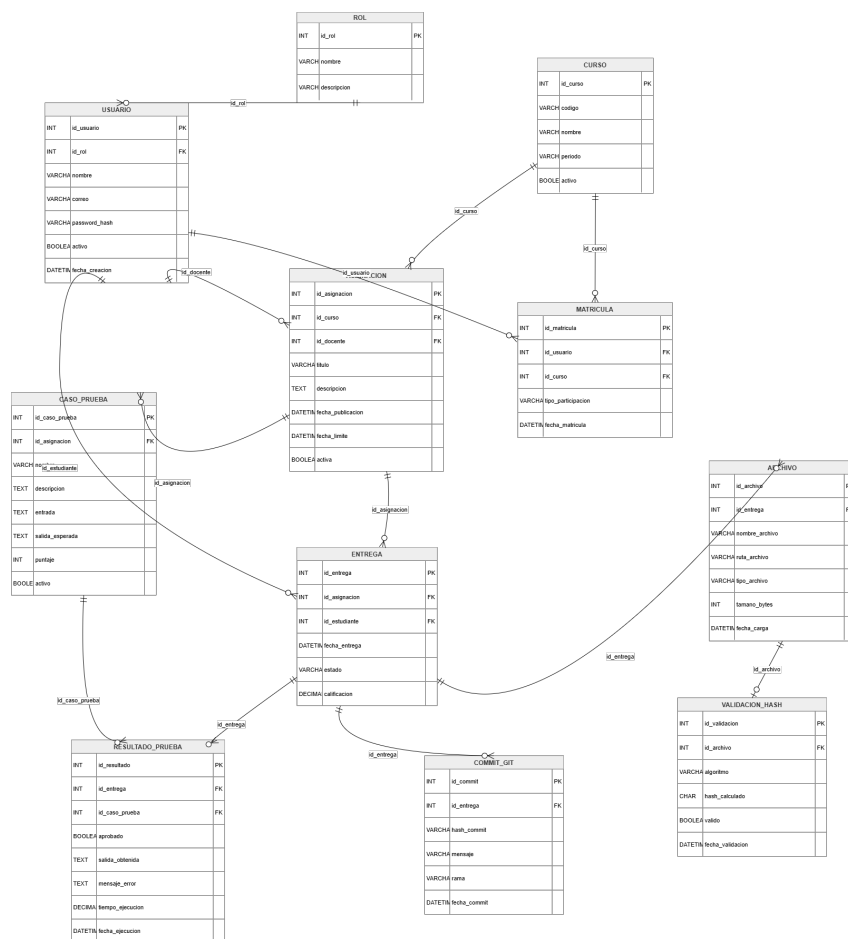


Figura 2.2: Modelo Relacional normalizado em 4FN

Esquema relacional

A continuación se presenta el esquema relacional del sistema:

Listing 2.1: Esquema relacional normalizado

```
1  ROL(
```

```

2      id_rol PK,
3      nombre UNIQUE NOT NULL,
4      descripcion
5  )
6
7  USUARIO(
8      id_usuario PK,
9      id_rol FK NOT NULL,
10     nombre NOT NULL,
11     correo UNIQUE NOT NULL,
12     password_hash NOT NULL,
13     activo NOT NULL,
14     fecha_creacion NOT NULL
15 )
16
17 CURSO(
18     id_curso PK,
19     codigo UNIQUE NOT NULL,
20     nombre NOT NULL,
21     periodo NOT NULL,
22     activo NOT NULL
23 )
24
25 MATRICULA(
26     id_matricula PK,
27     id_usuario FK NOT NULL,
28     id_curso FK NOT NULL,
29     tipo_participacion NOT NULL,
30     fecha_matricula NOT NULL,
31     UNIQUE(id_usuario , id_curso)
32 )
33
34 ASIGNACION(
35     id_asignacion PK,
36     id_curso FK NOT NULL,
37     id_docente FK NOT NULL,
38     titulo NOT NULL,
39     descripcion ,
40     fecha_publicacion NOT NULL,
41     fecha_limite NOT NULL,
42     activa NOT NULL
43 )
44
45 ENTREGA(
46     id_entrega PK,
47     id_asignacion FK NOT NULL,
48     id_estudiante FK NOT NULL,
49     fecha_entrega NOT NULL,
50     estado NOT NULL,
51     calificacion ,
52     UNIQUE(id_asignacion , id_estudiante)
53 )
54
55 ARCHIVO(
56     id_archivo PK,
57     id_entrega FK NOT NULL,
58     nombre_archivo NOT NULL,

```

```

59     ruta_archivo NOT NULL,
60     tipo_archivo NOT NULL,
61     tamano_bytes NOT NULL,
62     fecha_carga NOT NULL
63 )
64
65 VALIDACION.HASH(
66     id_validacion PK,
67     id_archivo FK UNIQUE NOT NULL,
68     algoritmo NOT NULL,
69     hash_calculado NOT NULL,
70     valido NOT NULL,
71     fecha_validacion NOT NULL
72 )
73
74 COMMIT.GIT(
75     id_commit PK,
76     id_entrega FK NOT NULL,
77     hash_commit UNIQUE NOT NULL,
78     mensaje ,
79     rama NOT NULL,
80     fecha_commit NOT NULL
81 )
82
83 CASO.PRUEBA(
84     id_caso_prueba PK,
85     id_asignacion FK NOT NULL,
86     nombre NOT NULL,
87     descripcion ,
88     entrada ,
89     salida_esperada NOT NULL,
90     puntaje NOT NULL,
91     activo NOT NULL
92 )
93
94 RESULTADO.PRUEBA(
95     id_resultado PK,
96     id_entrega FK NOT NULL,
97     id_caso_prueba FK NOT NULL,
98     aprobado NOT NULL,
99     salida_obtenida ,
100    mensaje_error ,
101    tiempo_ejecucion ,
102    fecha_ejecucion NOT NULL,
103    UNIQUE(id_entrega , id_caso_prueba)
104 )

```

Justificación de normalización en 4FN

El modelo se considera normalizado hasta la Cuarta Forma Normal debido a que cumple con las siguientes condiciones:

- Cada tabla representa una única entidad o relación del dominio.
- Todos los atributos dependen directamente de la clave primaria de su respectiva tabla.

- No existen dependencias parciales, ya que las relaciones de muchos a muchos se resuelven mediante tablas intermedias, como **MATRICULA**.
- No existen dependencias transitivas entre atributos no clave.
- No se almacenan listas, grupos repetitivos ni atributos multivaluados dentro de una misma tabla.
- Las dependencias multivaluadas fueron separadas en tablas independientes, como **ARCHIVO**, **COMMIT_GIT**, **CASO_PRUEBA** y **RESULTADO_PRUEBA**.

Por ejemplo, una entrega puede tener varios archivos, varios commits y varios resultados de prueba. Si esos datos se almacenaran directamente en la tabla **ENTREGA**, se generarían grupos repetidos y redundancia. Por esta razón, cada uno de esos elementos se modela como una tabla independiente relacionada mediante claves foráneas.

Diccionario de datos

Tabla: ROL

Descripción: Almacena los roles disponibles dentro del sistema, como estudiante, docente o administrador.

Cuadro 2.3: Diccionario de datos de la tabla ROL

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_rol	INT	Entero positivo	No	PK, AUTO_INCREMENT
nombre	VARCHAR(50)	ESTUDIANTE, DOCENTE, ADMINISTRADOR	No	UNIQUE, NOT NULL
descripcion	VARCHAR(255)	Texto descriptivo	Sí	Ninguna

Tabla: USUARIO

Descripción: Almacena la información de las personas que utilizan la plataforma.

Cuadro 2.4: Diccionario de datos de la tabla USUARIO

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_usuario	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_rol	INT	ID existente en ROL	No	FK, NOT NULL
nombre	VARCHAR(100)	Texto alfabético	No	NOT NULL
correo	VARCHAR(150)	Formato de correo válido	No	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	Hash de contraseña	No	NOT NULL
activo	BOOLEAN	TRUE, FALSE	No	DEFAULT TRUE
fecha_creacion	DATETIME	Fecha válida	No	DEFAULT CURRENT_TIMESTAMP

Tabla: CURSO

Descripción: Representa los cursos académicos disponibles en la plataforma.

Cuadro 2.5: Diccionario de datos de la tabla CURSO

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_curso	INT	Entero positivo	No	PK, AUTO_INCREMENT
codigo	VARCHAR(20)	Código de curso, por ejemplo IC6821	No	UNIQUE, NOT NULL
nombre	VARCHAR(100)	Nombre del curso	No	NOT NULL
periodo	VARCHAR(20)	I-2026, II-2026	No	NOT NULL
activo	BOOLEAN	TRUE, FALSE	No	DEFAULT TRUE

Tabla: MATRICULA

Descripción: Relaciona usuarios con cursos, indicando si participan como estudiantes o docentes.

Cuadro 2.6: Diccionario de datos de la tabla MATRICULA

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_matricula	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_usuario	INT	ID existente en USUARIO	No	FK, NOT NULL
id_curso	INT	ID existente en CURSO	No	FK, NOT NULL
tipo_participacion	VARCHAR(20)	ESTUDIANTE, DOCENTE	No	NOT NULL
fecha_matricula	DATETIME	Fecha válida	No	DEFAULT CURRENT_TIMESTAMP
Restricción adicional: UNIQUE(id_usuario, id_curso) para evitar que un mismo usuario se registre más de una vez en el mismo curso.				

Tabla: ASIGNACION

Descripción: Almacena las tareas, prácticas o proyectos creados por los docentes dentro de un curso.

Cuadro 2.7: Diccionario de datos de la tabla ASIGNACION

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_asignacion	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_curso	INT	ID existente en CURSO	No	FK, NOT NULL
id_docente	INT	ID existente en USUARIO	No	FK, NOT NULL
titulo	VARCHAR(150)	Texto descriptivo	No	NOT NULL
descripcion	TEXT	Texto largo	Sí	Ninguna
fecha_publicacion	DATETIME	Fecha válida	No	NOT NULL
fecha_limite	DATETIME	Fecha posterior a la publicación	No	NOT NULL
activa	BOOLEAN	TRUE, FALSE	No	DEFAULT TRUE

Tabla: ENTREGA

Descripción: Representa la entrega realizada por un estudiante para una asignación específica.

Cuadro 2.8: Diccionario de datos de la tabla ENTREGA

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_entrega	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_asignacion	INT	ID existente en ASIGNACION	No	FK, NOT NULL
id_estudiante	INT	ID existente en USUARIO	No	FK, NOT NULL
fecha_entrega	DATETIME	Fecha válida	No	DEFAULT CURRENT_TIMESTAMP
estado	VARCHAR(30)	RECIBIDA, VALIDADA, RECHAZADA, CALIFICADA	No	NOT NULL
calificacion	DECIMAL(5,2)	Valor entre 0.00 y 100.00	Sí	CHECK entre 0 y 100
Restricción adicional: UNIQUE(id_asignacion, id_estudiante) si únicamente se permite una entrega final por estudiante. Si se permiten varios intentos, se recomienda eliminar esta restricción y agregar el campo numero_intento .				

Tabla: ARCHIVO

Descripción: Almacena los archivos enviados como parte de una entrega.

Cuadro 2.9: Diccionario de datos de la tabla ARCHIVO

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_archivo	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_entrega	INT	ID existente en ENTREGA	No	FK, NOT NULL
nombre_archivo	VARCHAR(150)	Nombre del archivo	No	NOT NULL
ruta_archivo	VARCHAR(255)	Ruta física o lógica del archivo	No	NOT NULL
tipo_archivo	VARCHAR(10)	.py	No	NOT NULL
tamano_bytes	INT	Mayor que 0	No	CHECK > 0
fecha_carga	DATETIME	Fecha válida	No	DEFAULT CURRENT_TIMESTAMP

Tabla: VALIDACION_HASH

Descripción: Almacena el resultado de la validación criptográfica aplicada a cada archivo.

Cuadro 2.10: Diccionario de datos de la tabla VALIDACION_HASH

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_validacion	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_archivo	INT	ID existente en ARCHIVO	No	FK, UNIQUE, NOT NULL
algoritmo	VARCHAR(20)	SHA-256	No	NOT NULL
hash_calculado	CHAR(64)	Cadena hexadecimal SHA-256	No	NOT NULL
valido	BOOLEAN	TRUE, FALSE	No	NOT NULL
fecha_validacion	DATETIME	Fecha válida	No	DEFAULT CURRENT_TIMESTAMP

Tabla: COMMIT_GIT

Descripción: Registra los commits generados o asociados a una entrega en el sistema de control de versiones Git.

Cuadro 2.11: Diccionario de datos de la tabla COMMIT_GIT

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_commit	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_entrega	INT	ID existente en ENTREGA	No	FK, NOT NULL
hash_commit	VARCHAR(40)	Hash de commit Git	No	UNIQUE, NOT NULL
mensaje	VARCHAR(255)	Mensaje del commit	Sí	Ninguna
rama	VARCHAR(100)	Nombre de rama, por ejemplo main	No	NOT NULL
fecha_commit	DATETIME	Fecha válida	No	NOT NULL

Tabla: CASO_PRUEBA

Descripción: Almacena los casos de prueba definidos por el docente para evaluar automáticamente una asignación.

Cuadro 2.12: Diccionario de datos de la tabla CASO_PRUEBA

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_caso_prueba	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_asignacion	INT	ID existente en ASIGNACION	No	FK, NOT NULL
nombre	VARCHAR(100)	Texto descriptivo	No	NOT NULL
descripcion	TEXT	Explicación del caso	Sí	Ninguna
entrada	TEXT	Datos de entrada	Sí	Ninguna
salida_esperada	TEXT	Resultado esperado	No	NOT NULL
puntaje	INT	Mayor o igual a 0	No	CHECK ≥ 0
activo	BOOLEAN	TRUE, FALSE	No	DEFAULT TRUE

Tabla: RESULTADO_PRUEBA

Descripción: Almacena los resultados obtenidos al ejecutar los casos de prueba sobre una entrega.

Cuadro 2.13: Diccionario de datos de la tabla RESULTADO_PRUEBA

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id_resultado	INT	Entero positivo	No	PK, AUTO_INCREMENT
id_entrega	INT	ID existente en ENTREGA	No	FK, NOT NULL
id_caso_prueba	INT	ID existente en CASO_PRUEBA	No	FK, NOT NULL
aprobado	BOOLEAN	TRUE, FALSE	No	NOT NULL
salida_obtenida	TEXT	Resultado generado por el código	Sí	Ninguna
mensaje_error	TEXT	Error generado durante la ejecución	Sí	Ninguna
tiempo_ejecucion	DECIMAL(8,3)	Segundos, mayor o igual a 0	Sí	CHECK ≥ 0
fecha_ejecucion	DATETIME	Fecha válida	No	DEFAULT CURRENT_TIMESTAMP
Restricción adicional: UNIQUE(id_entrega, id_caso_prueba) para evitar duplicar resultados del mismo caso de prueba sobre la misma entrega.				

Conclusión del diseño de persistencia

El diseño de persistencia propuesto permite mantener separadas las responsabilidades del sistema y evita redundancia en la información almacenada. Las entregas, archivos, validaciones criptográficas, commits y resultados de prueba se modelan como entidades independientes, lo que facilita la trazabilidad del código entregado por los estudiantes. Además, el uso de claves primarias, claves foráneas y restricciones de unicidad permite mantener la integridad referencial

del sistema y asegurar que los datos almacenados sean consistentes con las reglas del dominio académico.

2.2. Diseño de Arquitectura

2.2.1. 2.1 Diagrama de Componentes

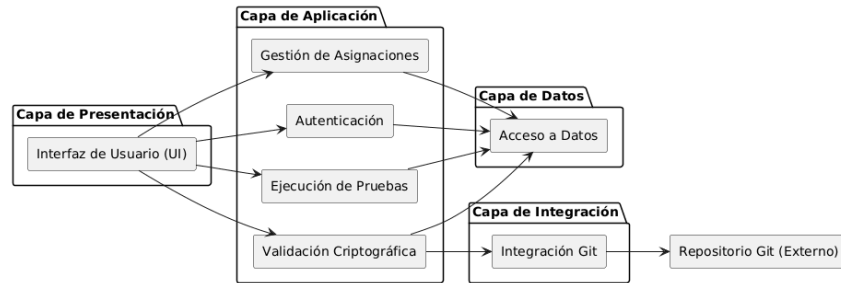


Figura 2.3: Diagrama de componentes del sistema

Descripción por componente (ordenado por capas)

Capa de Presentación Interfaz de Usuario (UI). *Rol:* Permite autenticación, carga de archivos y seguimiento de entregas. **Interfaces:** IUserSession, IAssignmentView. **Métodos:**

- Login(email: string, password: string) ->SessionToken *Ejemplo:* Login("estudiante@tec.cr",Clave123") ->.eyJhbGciOi...
- UploadFile(assignmentId: int, filePath: string) ->UploadResult *Ejemplo:* UploadFile(12,C:\proyecto\main.py") ->{status:."K", sha256:."8F..."}

Capa de Aplicación Autenticación. *Rol:* Verifica credenciales y emite tokens. **Interfaces:** IAuthService. **Métodos:**

- Login(email: string, password: string) ->Token *Ejemplo:* Login("docente@tec.cr",Clave2026") ->.eyJhbGciOi...
- ValidateToken(token: string) ->bool *Ejemplo:* ValidateToken(.eyJhbGciOi...) ->true

Validación Criptográfica. *Rol:* Genera y verifica hashes SHA-256 de los archivos fuente. **Interfaces:** IHashService. **Métodos:**

- ComputeSHA256(content: bytes) ->string *Ejemplo:* ComputeSHA256(<bytes>) ->.8F2C6..."
- VerifyHash(content: bytes, hash: string) ->bool *Ejemplo:* VerifyHash(<bytes>,.8F2C6...) ->true

Gestión de Asignaciones. *Rol:* Crea asignaciones y registra entregas. **Interfaces:** IAssignmentService, IAssignmentObservable. **Métodos:**

- `CreateAssignment(dto: AssignmentDTO) ->int` *Ejemplo: CreateAssignment({titulo:"Tarea 1"}) ->34*
- `RegisterDelivery(assignmentId: int, studentId: int, fileId: int) ->bool` *Ejemplo: RegisterDelivery(34,1021,778) ->true*
- `Subscribe(observer: IAssignmentObserver) ->void` *Ejemplo: Subscribe(EmailNotifier)*

Ejecución de Pruebas. *Rol:* Ejecuta casos de prueba automáticos por asignación. **Interfaces:** `ITestEngine`. **Métodos:**

- `ExecuteTests(assignmentId: int, entregaId: int) ->List<TestResult>`
Ejemplo: ExecuteTests(34,9002) ->[{case:"TC1", passed:true}]

Capa de Integración Integración Git. *Rol:* Sincroniza cambios y realiza commits. **Interfaces:** `IGitAdapter`. **Métodos:**

- `Commit(message: string, hashes: List<string>) ->string` *Ejemplo: Commit(.^{Entrega}entrega final",[.8F...]) ->ç1d2e3f"*
- `Pull() ->bool` *Ejemplo: Pull() ->true*

Capa de Datos Acceso a Datos. *Rol:* Abstrae la persistencia y operaciones transaccionales. **Interfaces:** `IRepository<T>`, `IUnitOfWork`. **Métodos:**

- `Add(entity: T) ->void` *Ejemplo: Add(Entrega{...})*
- `FindById(id: int) ->T` *Ejemplo: FindById(9002) ->Entrega{...}*
- `SaveChanges() ->int` *Ejemplo: SaveChanges() ->3*

2.2.2. 2.2 Diagrama de Clases

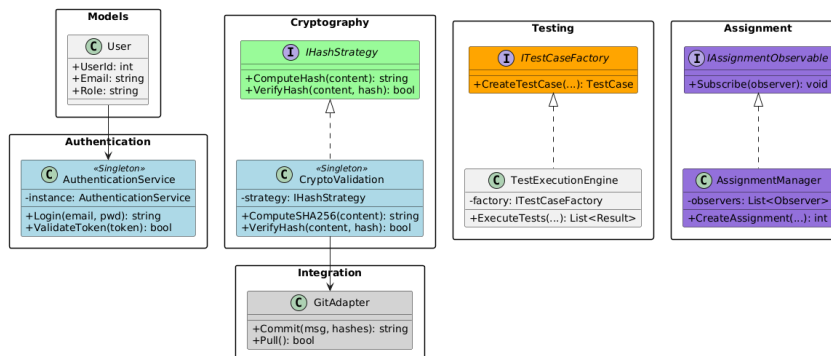


Figura 2.4: Diagrama de clases por paquetes con patrones de diseño

Patrones de diseño identificados (colores en el diagrama)

Colores: Azul = Singleton, Verde = Strategy, Naranja = Factory Method, Morado = Observer, Gris = Adapter.

Singleton (Azul) Se usa en servicios centrales para mantener una única instancia (ej. `AuthenticationService`, `CryptoValidation`). *Motivo:* evitar estados inconsistentes y duplicación de recursos. *Implementación:* instancia estática y método de acceso único.

Strategy (Verde) Se usa para seleccionar el algoritmo de hash. *Motivo:* cambiar el algoritmo sin modificar clientes. *Implementación:* interfaz `IHashStrategy` y una estrategia concreta (SHA-256).

Factory Method (Naranja) Se usa para crear casos de prueba según tipo de asignación. *Motivo:* centralizar la creación y evitar acoplamiento. *Implementación:* `ITestCaseFactory` crea instancias específicas.

Observer (Morado) Se usa para notificar eventos de asignación sin acoplar módulos. *Motivo:* permitir múltiples notificaciones (correo, UI, logs). *Implementación:* `IAssignmentObservable` y observadores suscritos.

Adapter (Gris) Se usa para encapsular la API de Git. *Motivo:* aislar dependencia externa y facilitar cambios. *Implementación:* `GitAdapter` traduce llamadas internas.

Capítulo 3

Conclusiones

El desarrollo de la arquitectura para la *Plataforma para la Enseñanza de Programación* permitió definir una solución estructurada para atender problemas relevantes dentro de los cursos introductorios de programación, especialmente en relación con la autoría del código, la trazabilidad de los cambios realizados por los estudiantes y la automatización parcial del proceso de revisión. A partir del análisis realizado, se evidencia que una plataforma académica de este tipo no debe limitarse únicamente a recibir archivos o mostrar asignaciones, sino que debe integrar mecanismos que permitan validar la integridad de las entregas, registrar evidencia técnica del proceso y facilitar al docente la evaluación de los trabajos recibidos.

En cuanto al diseño de persistencia, se logró construir un modelo de datos capaz de representar las principales entidades del sistema, tales como usuarios, roles, cursos, matrículas, asignaciones, entregas, archivos, validaciones criptográficas, commits de Git, casos de prueba y resultados de ejecución. La separación de estas entidades permite mantener una estructura clara y coherente, evitando la duplicación innecesaria de información y facilitando la administración de los datos. Además, la normalización hasta Cuarta Forma Normal contribuye a reducir dependencias multivaluadas y a garantizar que cada tabla represente una única responsabilidad dentro del dominio del sistema. Esto resulta especialmente importante en una plataforma donde una entrega puede tener varios archivos, distintos resultados de pruebas y registros asociados al control de versiones.

El modelo conceptual permitió identificar correctamente las relaciones entre los elementos del sistema y sus respectivas cardinalidades. Por ejemplo, se estableció que un curso puede contener múltiples asignaciones, que una asignación puede recibir muchas entregas y que cada entrega puede estar compuesta por uno o varios archivos. Estas relaciones reflejan de manera adecuada el funcionamiento de un entorno académico real. De igual forma, el modelo lógico permitió transformar dichas relaciones en una estructura relacional normalizada, incorporando claves primarias, claves foráneas, restricciones de unicidad y reglas de integridad que ayudan a preservar la consistencia de los datos almacenados.

Desde el punto de vista arquitectónico, la división del sistema en capas

facilita la comprensión y el mantenimiento de la solución. La separación entre la capa de presentación, la capa de aplicación, la capa de integración y la capa de datos permite distribuir responsabilidades de manera ordenada. La interfaz de usuario se encarga de facilitar la interacción con estudiantes y docentes; los servicios de aplicación gestionan procesos como autenticación, asignaciones, validación criptográfica y ejecución de pruebas; la capa de integración encapsula la comunicación con herramientas externas como Git; y la capa de datos abstrae el acceso a la persistencia. Esta organización favorece una solución menos acoplada y más fácil de modificar en el futuro.

El diagrama de componentes permitió visualizar cómo interactúan los principales módulos del sistema y qué interfaces publica cada uno de ellos. Esta representación resulta útil porque permite comprender el flujo general de la plataforma: el usuario inicia sesión, consulta o crea asignaciones, carga archivos, el sistema calcula hashes criptográficos, registra la entrega, sincroniza cambios mediante Git y ejecuta pruebas automáticas. Al documentar los métodos publicados por cada componente, junto con sus entradas, salidas y ejemplos, se obtiene una base clara para una futura implementación del sistema.

Por otra parte, el diagrama de clases permitió representar con mayor detalle la estructura interna de la solución. Al dividir los elementos en paquetes y aplicar principios de bajo acoplamiento, se favorece una arquitectura más mantenible y extensible. Esta separación permite que cambios en una parte del sistema, como el algoritmo de validación criptográfica o el mecanismo de integración con Git, no afecten directamente a otros módulos. Esto es fundamental en proyectos académicos y profesionales, ya que los sistemas suelen evolucionar con el tiempo y requieren adaptarse a nuevas necesidades sin comprometer su funcionamiento general.

La identificación de patrones de diseño también representa un aporte importante dentro de la arquitectura propuesta. El patrón *Singleton* permite controlar la existencia de servicios centrales que no deberían duplicarse innecesariamente. El patrón *Strategy* facilita cambiar el algoritmo de hash sin modificar el código cliente. El patrón *Factory Method* centraliza la creación de casos de prueba según el tipo de asignación, reduciendo el acoplamiento entre clases. El patrón *Observer* permite notificar eventos del sistema, como nuevas entregas o cambios de estado, sin depender directamente de los componentes receptores. Finalmente, el patrón *Adapter* permite encapsular la comunicación con Git, evitando que la lógica interna del sistema dependa directamente de una herramienta externa específica.

En conjunto, los artefactos desarrollados permiten comprobar que la solución propuesta tiene una base arquitectónica sólida. El diseño de persistencia garantiza una estructura de datos ordenada y consistente; el diseño de componentes define responsabilidades claras entre módulos; el diagrama de clases muestra una organización orientada a objetos coherente; y los patrones de diseño aportan flexibilidad, reutilización y mantenibilidad. Todo esto contribuye a que la plataforma pueda crecer en el futuro, incorporando nuevas funcionalidades como reportes de desempeño, análisis de similitud entre entregas, integración con otros repositorios o sistemas de notificación más avanzados.

En conclusión, la arquitectura propuesta combina un modelo de datos normalizado, una separación clara por componentes y el uso de patrones de diseño para construir una solución más ordenada y mantenible. La incorporación de herramientas como Git y funciones hash fortalece la trazabilidad y la integridad de las entregas, aspectos importantes dentro de una plataforma orientada a la enseñanza de programación (Silberschatz y cols., 2019; Chacon y Straub, 2014; Python Software Foundation, 2026).

Referencias

- Chacon, S., y Straub, B. (2014). *Pro git* (2.^a ed.). New York, NY: Apress.
Descargado de <https://git-scm.com/book/en/v2>
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Boston, MA: Addison-Wesley.
- Mermaid. (2026). Entity relationship diagrams [Manual de software informático]. Descargado de <https://mermaid.js.org/syntax/entityRelationshipDiagram.html>
- Python Software Foundation. (2026). hashlib — secure hashes and message digests [Manual de software informático]. Descargado de <https://docs.python.org/3/library/hashlib.html>
- Silberschatz, A., Korth, H. F., y Sudarshan, S. (2019). *Database system concepts* (7.^a ed.). New York, NY: McGraw-Hill Education.